

AI Engineer Applications - Problem Statement - Agentic KPI Builder

Jatin K Rai

+1-618-203-6737

Jatinkrai2002@gmail.com

Problem Statement:

Backend Engineer Applications - Problem Statement - Agentic KPI Builder Σ Inbox x



Sharmila V (HR) <sharmila@detecttechnologies.com>

to me, Aabidunisa, Renu ▼

2:52 PM (1 hour ago)



Dear Jatin,

Thank you for your interest in the role. Please find the attached problem statement for the next stage of our assessment process.

The assignment involves designing and building an Agentic KPI Builder, an AI-assisted workflow that allows a user to create custom KPIs through a guided, conversational interface. A synthetic dataset (operations_events.csv) is included for you to work with.

Please review the problem statement carefully before beginning. You are expected to submit your solution within 1 day of receiving this email.

Submission details:

- Share your solution as a ZIP file (Must be deployable through docker)
- Include a README with setup and run instructions
- Ensure the application runs locally with a single command (docker compose command)

If you have any questions regarding the problem statement, feel free to reach out.

We look forward to reviewing your submission.

© MIT

Requirement

KPI Name



KPI Name	Formula / Logic	Metric Type	Expected Value
Issue Count	Count events where <u>event_type</u> = Issue	count	99 (from full dataset)
Near Miss Count	Count events where <u>event_type</u> = Near Miss	count	103
Open Issue Count	Count where <u>event_type</u> = Issue AND <u>event_status</u> = Open	count	26
High Priority Issue Count	Count where <u>event_type</u> = Issue AND <u>priority</u> = High	count	25
Near Miss to Issue Ratio	<u>COUNT</u> (Near Miss) / <u>COUNT</u> (Issue)	ratio	1.04
Average Processing Hours	<u>AVG</u> (<u>processing_hours</u>) where <u>event_type</u> = Issue	average	8.34 hrs
Total Cost Impact	<u>SUM</u> (<u>cost_amount</u>) where <u>event_type</u> = Issue	sum	\$401,580
Resolution Rate	<u>COUNT</u> (<u>resolution_status</u> = Resolved) / <u>COUNT</u> (<u>event_type</u> = Issue)	ratio	Varies
Critical Issue Count	Count where <u>event_type</u> = Issue AND <u>priority</u> = Critical	count	Varies
<u>Avg</u> Customer Impact	<u>AVG</u> (<u>customer_count</u>) where <u>impact_type</u> = Customer AND <u>event_type</u> = Issue	average	Varies

Analysis:

KPIName → Formula/Logic → Metric Type → Expected Value

Agentic Flow & State Management

Q1

Focus: Agentic Flow & State Management

Design and implement the end-to-end agentic flow. A user enters a KPI name and the system guides them step by step to a computed result. Your implementation must cover:

- a) How the agent interprets the KPI name, confirms its understanding with the user, and generates 3-4 contextual multiple-choice questions to collect the definition.
- b) How the agent converts the user's answers into a structured KPI Intent (JSON). Show an example intent for 'Near Miss to Issue Ratio' with all required fields: metric_type, filters, aggregation, and time_period.
- c) How conversation state is tracked across steps - what data is stored and how the flow progresses from one step to the next.

Note: This is the core of the assignment. A working flow with at least one complete KPI path from name entry to computed value is the minimum expected outcome.

Analysis:

Assumptions:

1. pre-defined KPIs name in the memory of Agentic Application.
2. Each KPIs name with step by step computed result.
3. a. LLM will be used to generate 3-4 contextual multiple choice questions.
3. b. ~~SQL statement~~/pyspark python based code with predefined logics.
3. c . Long term memory will be used in database.

To implement an **agentic workflow** with LangGraph and LangChain primitives (@tool, @static, @prompt), it can model each of your KPI system functions as **nodes** in a graph. This gives us orchestration, self-healing, and traceability.

Validation & Calculation Engine

Q2

Focus: Validation & Calculation Engine

Implement the backend layer that validates a KPI Intent and executes it against the dataset. Your implementation must handle:

- a) Column and value validation - reject intents that reference columns or filter values that do not exist in operations_events.csv, and return a clear error message identifying the invalid field.
- b) KPI calculation - execute count, sum, average, and ratio aggregations by interpreting the intent JSON programmatically (no raw SQL). Handle divide-by-zero for ratio KPIs.
- c) Return a result object containing: the computed value, a human-readable formula string, and for ratio KPIs - the numerator and denominator values separately.

Analysis:

1. KPI intent and executes septs in the datasets.
2. Python code to reject the columns and value validation.
3. Python code to avoid the divide by zero.
4. Computed value, human readable using LLMs, etc...

How your submission will be evaluated

Assessment Dimension	What We Are Looking For
Agentic Flow	Does the agent guide the user step by step from KPI name to computed value? Are transitions between steps smooth?
Dynamic Questions	Do the questions adapt to the KPI name entered? Do they avoid being generic or repetitive?
Intent & Schema	Is the KPI definition represented as structured JSON? Are <u>metric_type</u> , filters, and <u>aggregation</u> correct?
KPI Calculation	Is the KPI value computed correctly from the CSV? Are count, sum, average, and ratio types handled?
Review Card UX	Is the review card clear and usable? Does the Approve button correctly reflect validation state?
Code Quality	Is the code clean and modular? Are concerns separated appropriately?

Analysis:

Agentic Flow ➔ Approach 2 -- in progress
Dynamic Question ➔ Done
Intent & Schema ➔ Done
KPI Calculation ➔ Done
Review Card UX ➔ Done
Code Quality ➔ Excellent with comments

What to include and how to deliver

Analysis:

Required Deliverables

- Working application (GitHub repo or ZIP). Include a README with setup instructions.
- The application must run locally with a single command (e.g. `npm start`, `python app.py`).
- The `operations_events.csv` file must be bundled or the README must explain how to add it.
- No paid API keys required to run the app. If using an LLM, provide a mock/rule-based fallback mode.
- If you made assumptions about the requirements, document them clearly in the README.
- If you ran out of time, leave commented-out stubs or a written explanation of how you would have implemented the remaining parts.

Zip level with deployment script

```
python -m streamlit run app.py --server.address 127.0.0.1 --server.port 8080
```

Readme file has instruction.

Env file has the keys.

Assumptions are mentioned.

Approach 2 and 3 of some codes are implemented. But partially.

Problem Statement:



Backend Engineer Applications - Problem Statement - Agentic KPI Builder

Inbox x



Sharmila V (HR) <sharmila@detecttechnologies.com>

to me, Aabidunisa, Renu

2:52 PM (1 hour ago)



Dear Jatin,

Analysis:

Thank you for your interest in the role. Please find the attached problem statement for the next stage of our assessment process.

The assignment involves designing and building an Agentic KPI Builder, an AI-assisted workflow that allows a user to create custom KPIs through a guided, conversational interface. A synthetic dataset (operations_events.csv) is included for you to work with.

Please review the problem statement carefully before beginning. You are expected to submit your solution within 1 day of receiving this email.

Submission details:

- Share your solution as a ZIP file (Must be deployable through docker)
- Include a README with setup and run instructions
- Ensure the application runs locally with a single command (docker compose command)

If you have any questions regarding the problem statement, feel free to reach out.

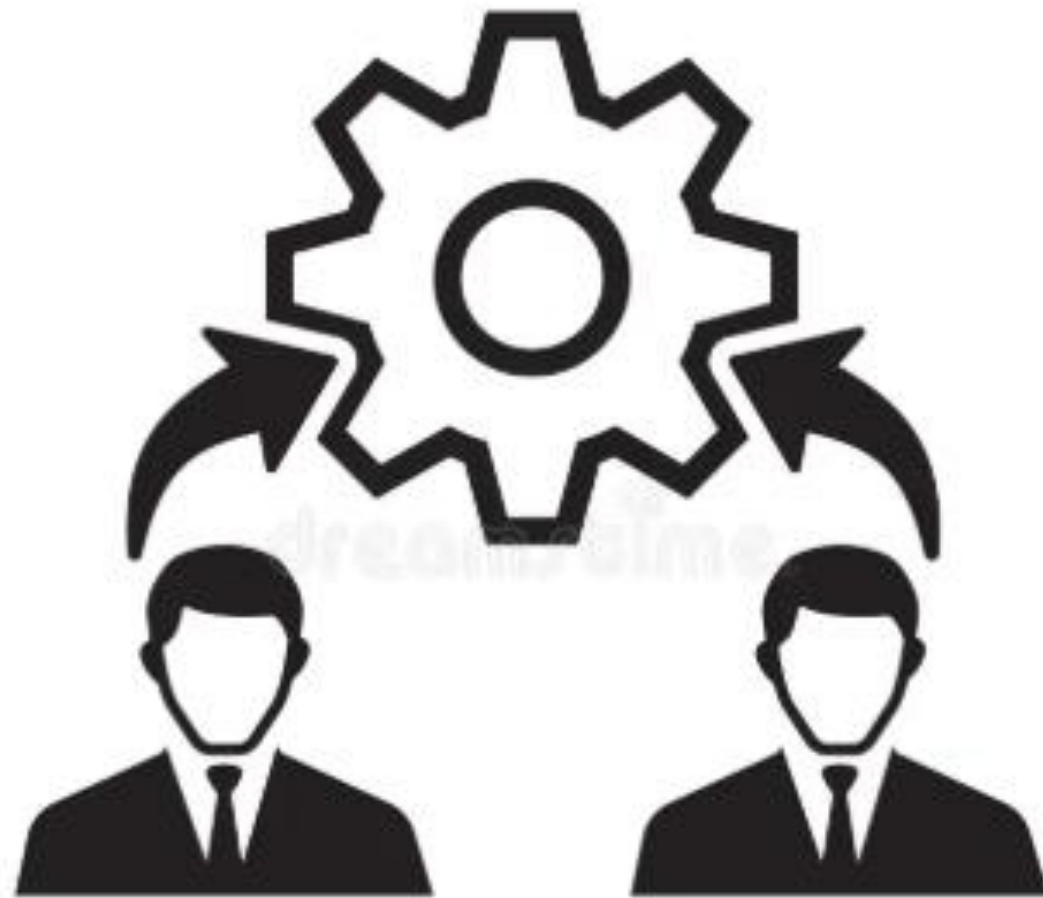
We look forward to reviewing your submission.

Completed with
dockerfile

As per Rohith – 2
days with simple
approach.

Completed with
docker compose
with AKS also.

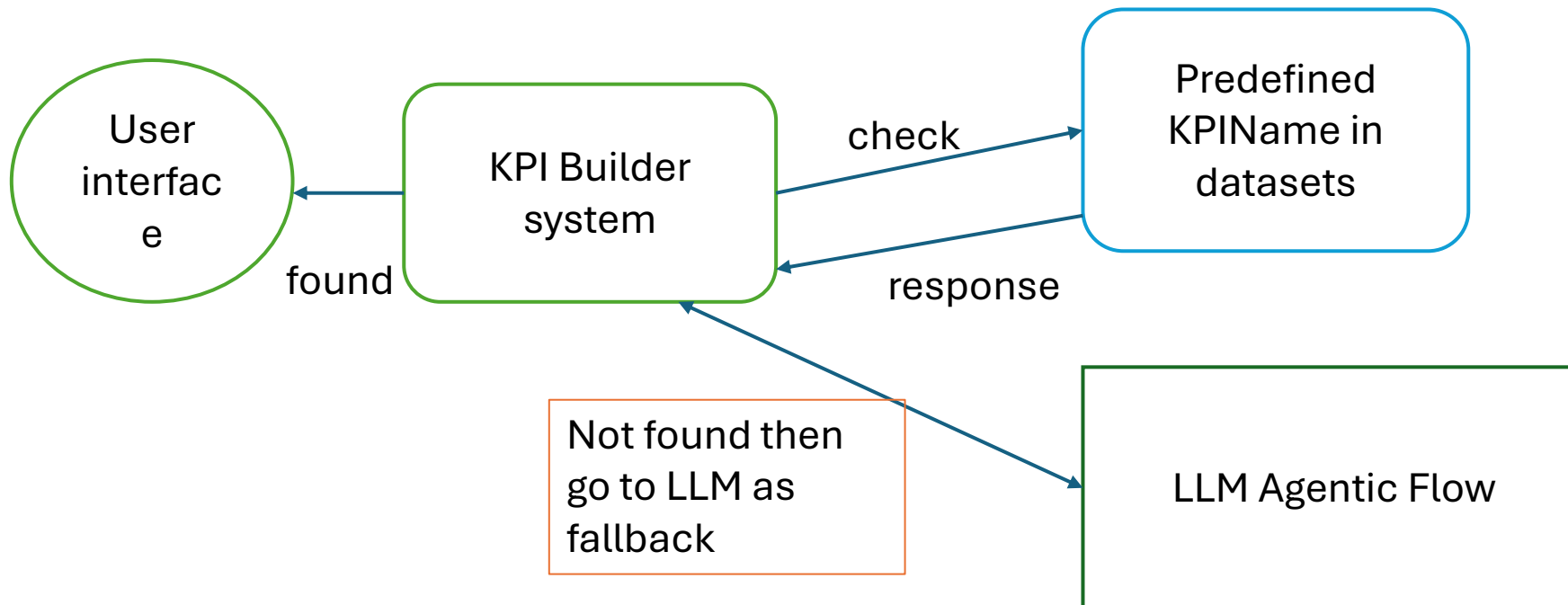
Completed with readme.



APPROACH

Approach 1:

1. User provides KPI Name using Conversational UI interface.
2. KPI Builder System will validate the KPI Name along with pre-defined KPIName in the datasets.
3. Provide response as Review card Interface.
4. In case, not found the in datasets, goto Agentic Flow – tool calling method to retrieve the information from LLM and provide response as Review card interface.

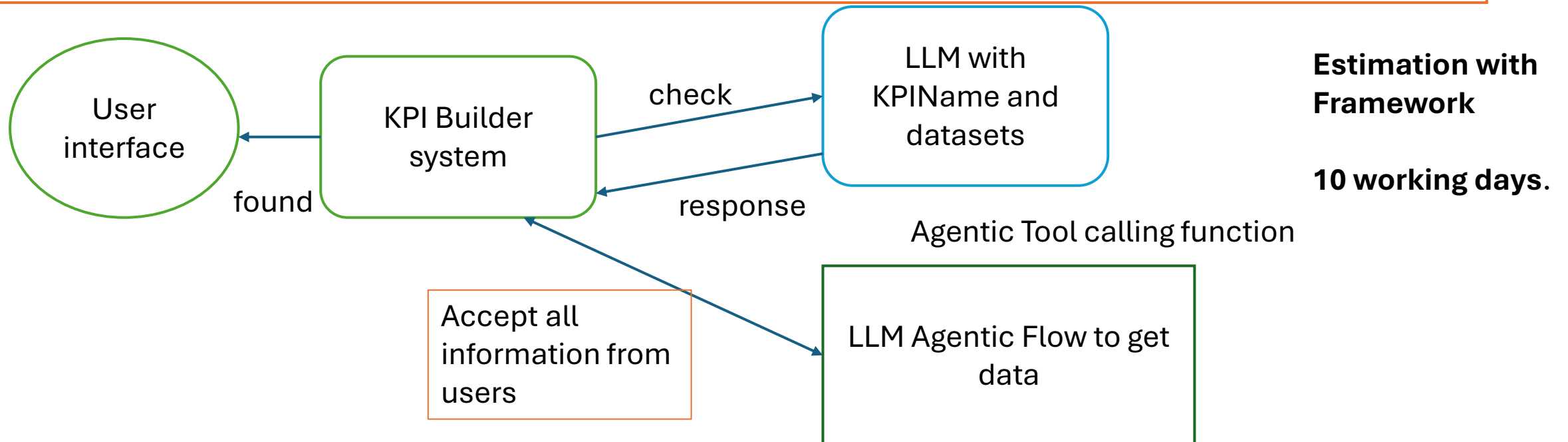


Estimation with Framework

5 working days.

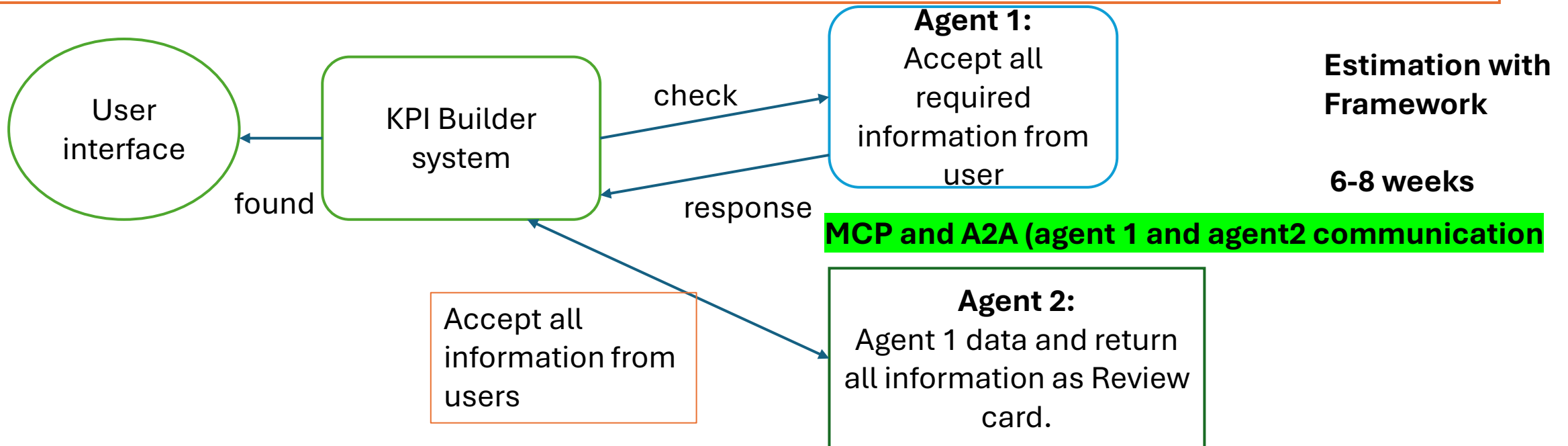
Approach 2:

1. User provides KPI Name using Conversational UI interface.
2. KPI Builder System will use that KPI Name with LLM along with predefined dataset.
3. KPI Builder system will ask more questions from user.
4. Provide response as Review card Interface.
5. In case, not found the in datasets, goto Agentic Flow – tool calling method to retrieve the information from LLM and provide response as Review card interface.



Approach 3: MCP and A2A

1. User provides KPI Name using Conversational UI interface.
2. KPI Builder System will use that KPI Name with LLM along with predefined dataset.
3. KPI Builder system will ask more questions from user.
4. Provide response as Review card Interface.
5. In case, not found the in datasets, goto Agentic Flow – tool calling method to retrieve the information from LLM and provide response as Review card interface.





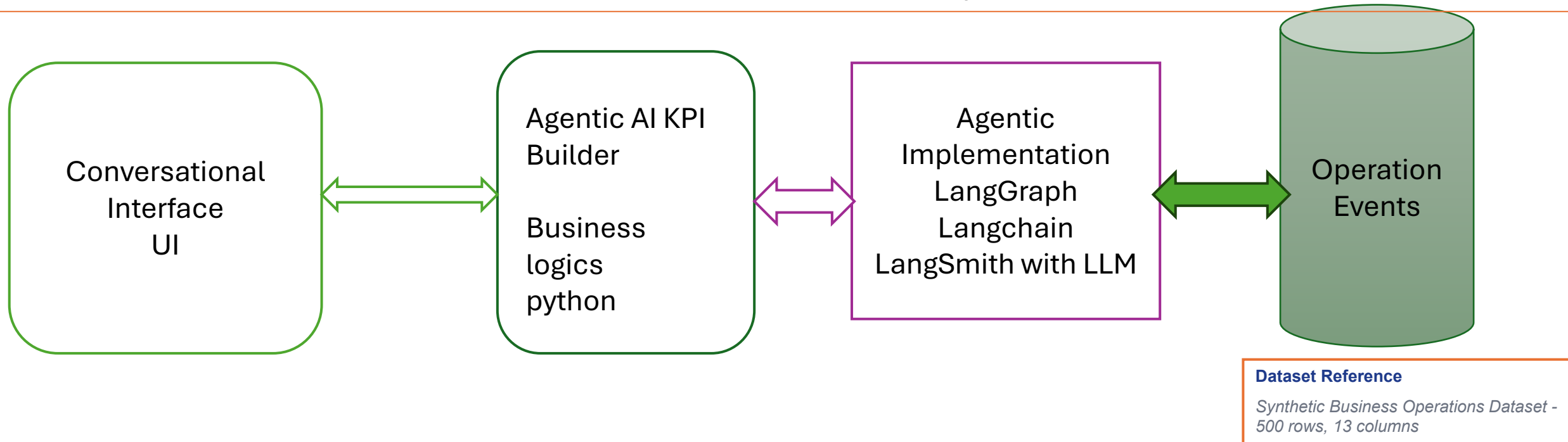
SOLUTION

Technologies stack: Agentic KPI Builder

The dataset is provided separately as operations_events.csv.

You may use any language, framework, or LLM integration approach.

- ➔ Using Python as language
- ➔ Langchain, Langgraph and LangSmith as framework
- ➔ Groq as LLM Models



Q3

Focus: Review Card UI

Build the KPI Review Card displayed after the calculation completes. The card must show:

- a) KPI Name, Description, Formula (human-readable), Computed Value, and Unit.
- b) Three action buttons: Approve, Edit (returns user to the question flow), and Regenerate (restarts from the KPI name entry).
- c) A status badge - Good, Warning, or Critical - driven by simple threshold rules you define. The Approve button must be disabled if the KPI failed validation.

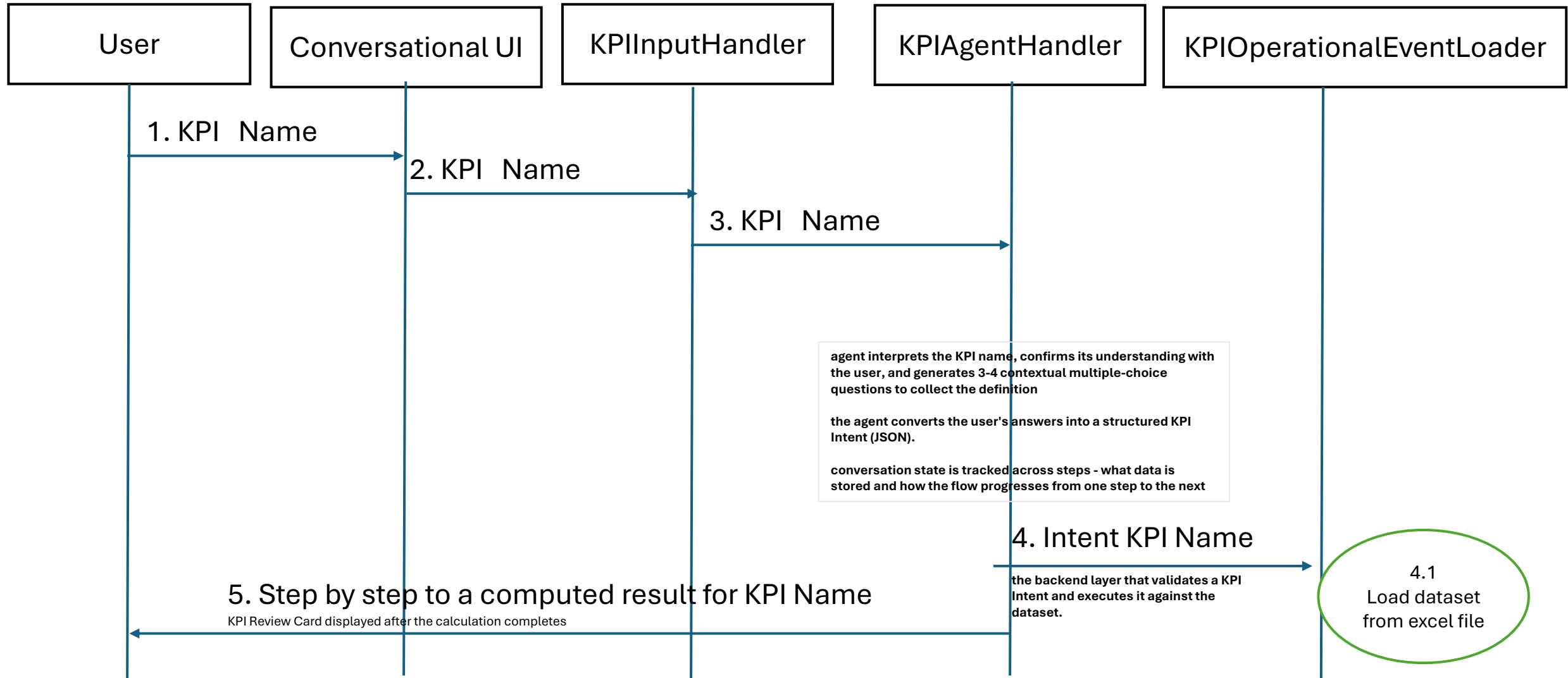
KPI Review Card

➔ The Approve button must be disabled if the KPI failed validation.



Design

Sequence diagram: (high level)



Class Diagram (high level):

KPINameReference

Class

KPI Name should be stored
with logics, metric type

ReviewCardGenerator

Class

KPI Review Card Generated

UI should have

Validation methods

Such as

1. The Approve button must be disabled if the KPI failed validation..

KPI Name, Description, Formula (human-readable), Computed Value,
and Unit.

Three action buttons: Approve, Edit (returns user to the question flow),
and Regenerate (restarts from the KPI name entry).

A status badge - Good, Warning, or Critical - driven by simple threshold
rules you define.

KPIOperationalEventLoader

Class

1. KPI name and steps calculation.

Validation methods

Such as

1. Near Miss and Opportunity events always have cost_amount = 0 by design.
2. When a KPI involves a ratio, always verify the denominator is not zero before computing.
3. The customer_count column is only meaningful when impact_type = Customer or Commercial.

Class Diagram: High level

ConversationalUI

Class

Accept input from the user as
KPI Name
Save conversational history.

KPIInputHandler

Class

Verify KPI Name from the KPI
Reference and if available return
otherwise calculate

KPIAgentHandler

Class

1. KPI name and steps calculation.
2. Store data
3. Maintain short terms and long term history.

Class Diagram: High level

ConversationalUI

Class

Accept input from the user as

KPI Name

Save conversational history.

KPIUserFeedbackHandler

Class

Accept User feedback

KPIPromptHandler

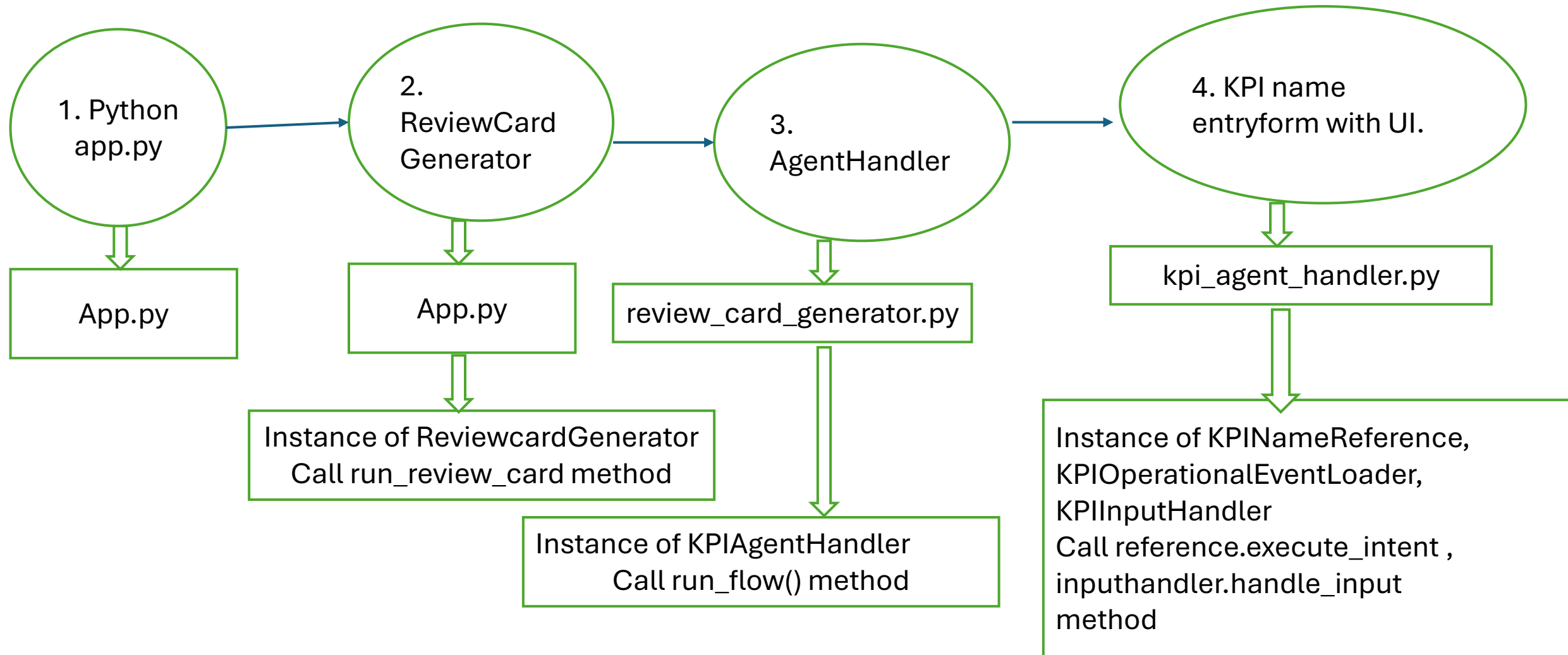
Class

1. KPI name with prompt.

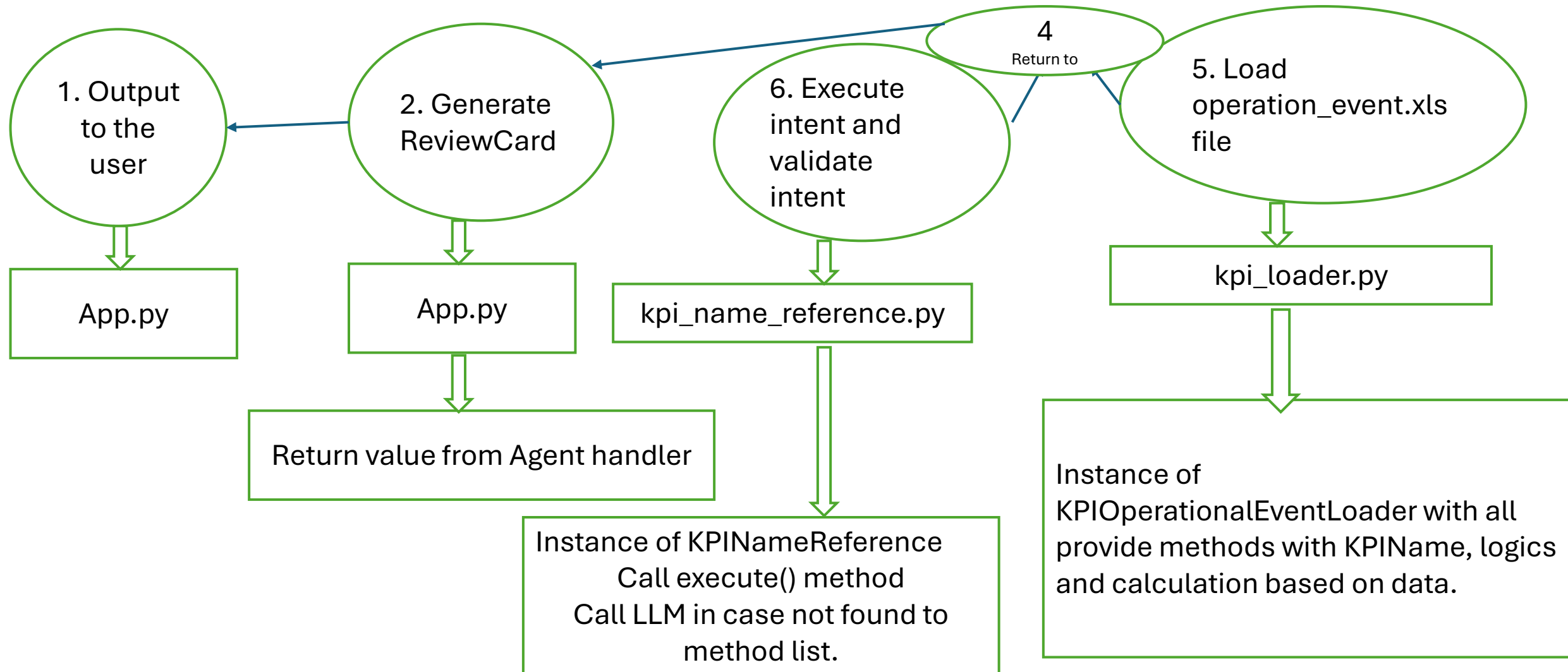
2. Store data

3. Maintain short terms and long term history.

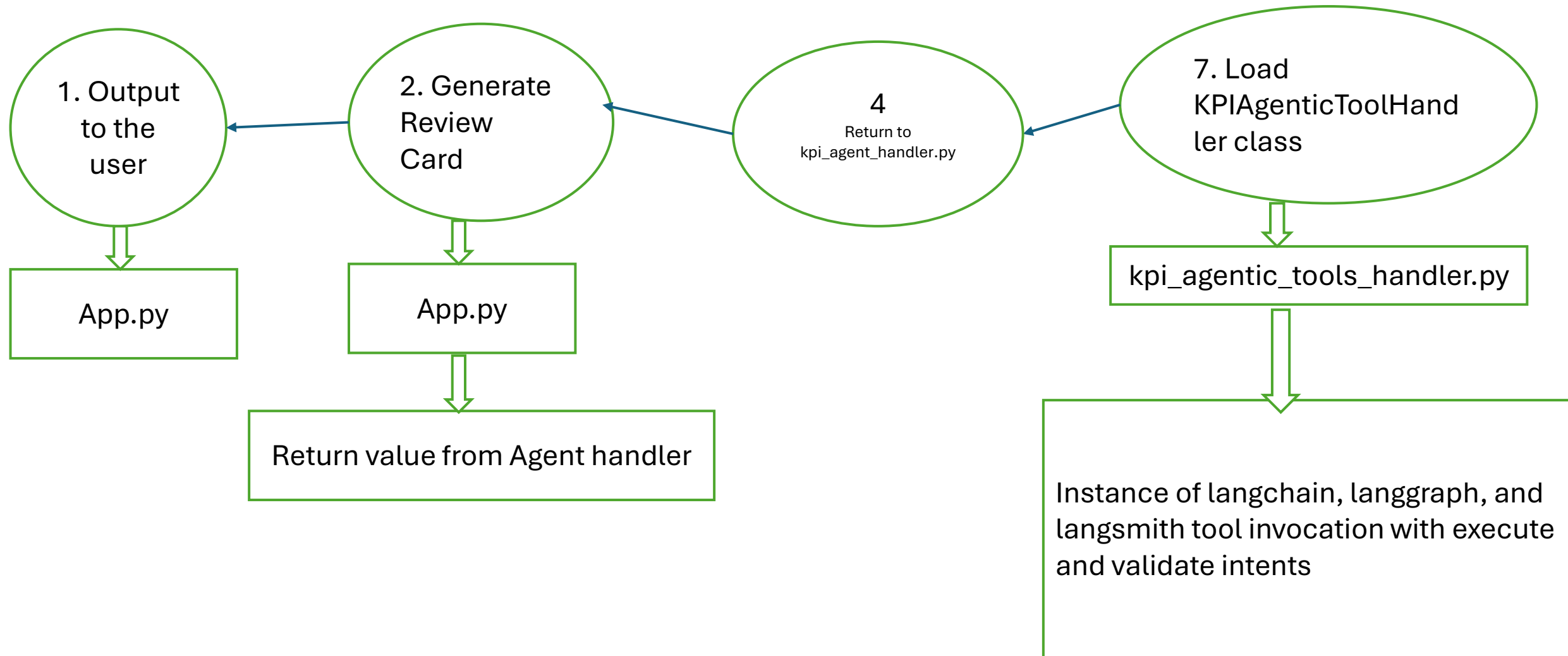
Flow Diagram: high level



Flow Diagram : High level



Flow Diagram: High level





Logging

Logging : Used Langsmith

```
api_key = os.getenv("LANGSMITH_API_KEY")
os.environ["LANGSMITH_TRACING"] = "true"
if not api_key:
    st.error("LANGSMITH_API_KEY not found. Please set it in .env.")
    return
```

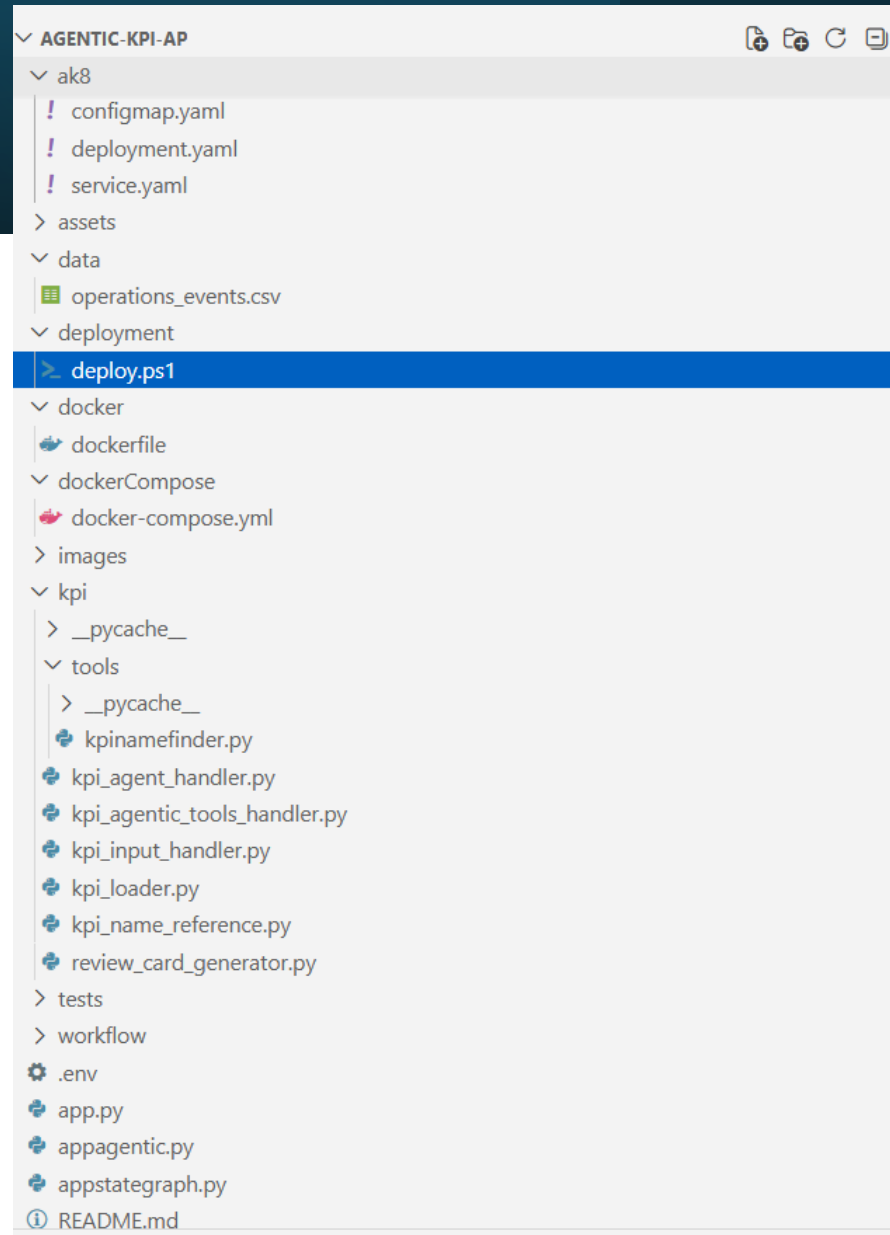
Extend to local logging text file with python log file.





Implementation

Directory Structure:



Unit Test case:



```
tests > test_kpi_system.py > test_near_miss_to_issue_ratio
1 import pytest
2 import pandas as pd
3 from kpi.kpi_loader import KPIOperationalEventLoader
4 from kpi.kpi_name_reference import KPINameReference
5
6 @pytest.fixture
7 def sample_loader(tmp_path):
8     # Create a temporary CSV file with sample data
9     data = pd.DataFrame({
10         "event_id": [1,2,3,4],
11         "event_date": ["2024-01-01", "2024-01-02", "2024-01-03", "2024-01-04"],
12         "business_unit": ["A", "A", "B", "B"],
13         "location": ["X", "Y", "X", "Y"],
14         "event_type": ["Issue", "Near Miss", "Issue", "Issue"],
15         "event_status": ["Open", "Closed", "Open", "Closed"],
16         "category": ["Cat1", "Cat2", "Cat1", "Cat2"],
17         "priority": ["High", "Low", "High", "Low"],
18         "impact_type": ["Minor", "Major", "Minor", "Major"],
19         "resolution_status": ["Resolved", "Pending", "Resolved", "Pending"],
20         "cost_amount": [100, 200, 300, 400],
21         "processing_hours": [5, 10, 15, 20],
22         "customer_count": [1, 2, 3, 4]
23     })
24     csv_file = tmp_path / "operations_events.csv"
25     data.to_csv(csv_file, index=False)
26     return KPIOperationalEventLoader(csv_file)
27
28 def test_issue_count(sample_loader):
29     reference = KPINameReference(sample_loader)
30     intent = {
31         "kpi_name": "IssueCount",
32         "metric_type": "Count",
33         "filters": "Only Issues",
```

Integration Test case:



User Acceptance Test case:



Deployment script:



```
# Deployment script for Agentic KPI Builder
```

```
$AppName = "agentic-kpi-app"
```

```
$ZipName = "$AppName.zip"
```

```
Write-Host "Cleaning old build..."
```

```
Remove-Item $ZipName -ErrorAction SilentlyContinue
```

```
Write-Host "Creating deployment package..."
```

```
Compress-Archive -Path `
```

```
    app.py,`
```

```
    appagentic.py,`
```

```
    appstategraph.py,`
```

```
    .env,`
```

```
    requirements.txt,`
```

```
    README.md,`
```

```
    data/operations_events.csv,`
```

```
    kpi/*.py,`
```

```
    kpi/tools/*.py,`
```

```
    assets/styles.css,`
```

```
    deployment\deploy.ps1,`
```

```
    tests/*.py,`
```

```
    workflow/*.py,`
```

```
    workflow/*.json,`
```

```
    workflow/*.yaml,`
```

```
    images/*.png`
```

```
    docker/*.*
```

```
    dockerCompose/docker-compose.yml`
```

```
    ak8/*.yaml`
```

```
    -DestinationPath $ZipName
```

```
Write-Host "Deployment package created: $ZipName"
```


Production check:



Post Production support:



Approach 2:



Approach 3:



Thanks:

*Thank
you!*